UVM Digital library - Shift Register Final Report

Shawn Chang
VIP 37920
chang950@purdue.edu

Spring 2026

The feedback for the report is to be returned to:

Shawn Chang
chang950@purdue.edu

This project was mentored by the following Teaching Assistants:

Ayush Singh, sing2570@purdue.edu
Anas Abughali, aabughal@purdue.edu

UVM Digital library - Shift Register Final Report

Shawn Chang
VIP 37920
chang950@purdue.edu

Abstract

In modern digital systems, shift registers support data alignment, serialization, and buffering, so verification must remain correct and scalable as features such as shift direction, parallel load, and reset behavior are added. However, a simple directed SystemVerilog testbench (even with assertions) can miss corner cases and does not scale well when the design grows or when similar modules (ex: FIFOs) require verification. This project addresses that gap by building a reusable, constrained-random verification environment for a shift register using the Universal Verification Methodology (UVM) rather than a simple SystemVerilog testbench.

The key idea is to take advantage of UVM's modular architecture. The Interface connects the DUT to the testbench, the driver drives signals into the DUT based on the sequencer's transactions, the monitor observes signals and converts activities into transactions, and the scoreboard validates the correct output based on the predictor inside the UVM environment. This structure enables systematic verification of both parallel-to-serial and serial-to-parallel behaviors across many randomized scenarios, while keeping the environment reusable to future designs. A core challenge is defining transactions and a cycle-accurate predictor that models the shift behavior correctly, then ensuring the monitor samples signals at the right times to avoid timing issues during scoreboard comparisons.

The measurable outcome is coverage, which is collected by the covergroups/coverpoints. The goal is to achieve 100 percent functional coverage, demonstrating that all planned scenarios in the test plan are exercised and checked. Limitations include the effort required to design the testbench. A simple module such as a shift register, still needs the extensive UVM design in order to get a basic testbench up and running, while a SystemVerilog testbench is easier, and faster to set up. Next, this UVM framework will be adapted to verify a FIFO by reusing the same architectural components (sequences, monitors, and scoreboards), demonstrating how verification reuse reduces development time and increases confidence in larger digital subsystems.

1 Introduction and Background

The Shift Register is a fundamental sequential logic component used for temporary data storage and controlled bit movement in digital systems. Shift registers often operate on clock edges and include control signals such as shift enable and active-low reset. Because of this, a single mistake within the component can result in data corruption, incorrect bit propagation, loss of synchronization, and overall system malfunction. Therefore, thorough verification is required to ensure reliable operation under both normal and corner-case conditions.

The reason SoCET has decided to use the Universal Verification Methodology (UVM) is due to its structured, modular, and reusable architecture. Instead of rewriting large testbenches after completing each design, UVM allows engineers to reuse existing testbench components such as drivers, monitors, scoreboards, and even agents across multiple projects. This significantly reduces development time, as well as allows an extremely scalable verification environment for complex digital systems.

2 Design

2.1 Top Level Architecture

The top-level UVM architecture for the Shift Register is shown in Figure 1. Due to the simplicity of the DUT, a single-agent UVM environment was selected. The Shift Register contains only one interface and a small amount of control signals, therefore, only one Sequencer, Driver, and Monitor were required. A multi-agent architecture was considered unnecessary because it would increase development effort without improving verification functional coverage.

The Sequencer and Sequence generate abstract test scenarios known as transactions and forward them to the Driver. The implemented sequence executes approximately 100 randomized multidirectional bit-shifting tests with randomized values for shift enable, active-low reset, and serial input. Randomized constrained testing was chosen over purely directed testing because it increases scenario diversity, as well as improving the probability of detecting corner-case failures.

The Driver converts transaction-level stimulus into pin-level signal activity through the interface. The Interface provides connectivity between the Driver, Monitor, and DUT while simplifying signal management.

The Monitor passively observes DUT activity and captures both applied inputs and resulting outputs into transaction objects. These transactions are forwarded to the Scoreboard.

The Scoreboard serves as the primary checking component. It compares actual DUT outputs captured by the Monitor against expected outputs generated by the Predictor. The Predictor acts as a golden reference model and computes correct shift register behavior based on transaction inputs.

This modular UVM environment improves debuggability through structured reporting phases and improves scalability through reusable verification components. Although effective, UVM introduces substantial development overhead. For a DUT as simple as a Shift Register, a conventional SystemVer-

ilog testbench could provide equivalent functional verification with lower implementation effort. However, UVM was selected to demonstrate industry-standard verification methodology and future scalability/reusability.

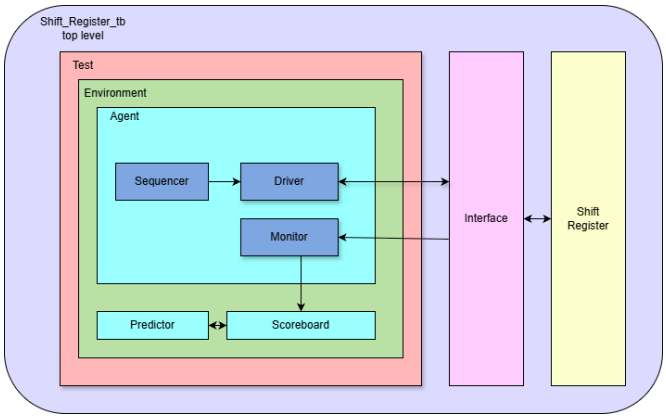

Figure 1: Shift Register Top Level Diagram

2.2 Deeper Dive into Code

2.2.1 Interface

Figure 2 down below captures a snippet of the Shift Register IF. The Interface, as stated above, sends signals to and from the DUT to the testbench, using modports. All signals can be seen through each instantiation of the modport, DUT, being the signals inputted inside the DUT, while tester, being the I/O from the testbench.

```
modport tester
(
    input clk, parallel_out, serial_out,
    output parallel_in, nRST, shift_en, serial_in, parallel_load

);
modport dut
(
    input clk , parallel_in, nRST, shift_en, serial_in, parallel_load,
    output parallel_out, serial_out
);
```

Figure 2: Shift Register Interface

2.2.2 Driver

Figure 3 below captures a snippet of the Driver. In the driver, the run phase is responsible for continuously receiving transactions from the sequencer. It then, pushes them to the DUT.

Inside the forever loop, the driver calls `seq_item_port.get_next_item(req_item)` to recieve the next randomized transaction. This transaction contains random values for `nRST`, `shift_enable`, and `serial_in`. The driver, then waits until the next negative edge of the clock, and drives these values into the virtual interface connected to the DUT. The reason we want to drive on the negative edge of the clock is because it allows inputs to stabilize before the DUT samples them on the following positive edge.

After the positive edge of the clock, the driver calls `seq_item_port.item_done()` to notify the sequencer that the transaction has completed, and the next item can be sent.

```
task run_phase(uvm_phase phase);
  transaction#(4) req_item;

  DUT_reset();

  forever begin
    seq_item_port.get_next_item(req_item);

    `uvm_info("DRV", $sformatf("Driving item %s", req_item.convert2str()), UVM_HIGH)

    @(negedge vif.clk);
    vif.nRST = req_item.nRST;
    vif.shift_en  = req_item.shift_enable;
    vif.serial_in = req_item.serial_in;

    @(posedge vif.clk);
    seq_item_port.item_done();
  end
endtask: run_phase
```

Figure 3: Shift Register Driver

2.2.3 Monitor

Figure 7 captures the essence of the Predictor. The Predictor is responsible for generating the expected output of the shift register.

In this design, the Predictor extends `uvm_subscriber`, which allows it to receive transactions through the `write()` function. The main variable used is `model_q`, which represents the Predictor's golden reference model of the Shift Register. On every transaction, the Predictor updates this variable based on the reset and shift enable signals.

If `nRST` is low, the Predictor accurately models the DUT's reset behavior by resetting the output to all ones. If reset is not active and `shift_enable` is high, the Predictor shifts in `serial_in`. The direction of the shift depends on the `SHIFT_MSB` parameter, and the size of `parallel_out` depends on the `NUM_BITS` parameter.

After updating its internal golden reference model, the Predictor assigns `model_q` to `parallel_out` and sends this expected transaction to the Scoreboard through `predictor_ap`.

2.2.4 Scoreboard

Figures 5 and 6 below captures a snippet of the Scoreboard. The main function of the scoreboard is to compare expected output by the predictor, against actual output from DUT.

In this design, the scoreboard uses two analysis implementation ports `exp_imp` and `act_imp`, to receive two separate streams of the transactions.

The `exp_imp` port receives the expected transaction from the predictor and stores it inside the queue named `exp_q`. These queues are vital for the scoreboard because the expected and actual transactions might not come at the same time. The scoreboard then, stores them temporarily until both are available.

Once each queue contains a transaction item, the Scoreboard removes the oldest expected/actual transaction using

```
    virtual shift_register_if#(NUM_BITS) vif;
    uvm_analysis_port #(transaction#(NUM_BITS)) sr_ap;
    uvm_analysis_port #(transaction#(NUM_BITS)) result_ap;

    function new(string name, uvm_component parent = null);
        super.new(name, parent);
    endfunction

    virtual function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        sr_ap = new("sr_ap", this);
        result_ap = new("result_ap", this);
        if (!uvm_config_db #(virtual shift_register_if#(NUM_BITS))::get(this, "", "shift_register_if", vif)) begin
            `uvm_fatal("MON", "Virtual interface not found")
        end
    endfunction

    virtual task run_phase(uvm_phase phase);
        transaction#(NUM_BITS) tx;
        transaction#(NUM_BITS) tx_result;
        forever begin
            @(posedge vif.clk);

            // Send full state to predictor
            tx = transaction#(NUM_BITS)::type_id::create("tx");
            tx.nRST = vif.nRST;
            tx.serial_in = vif.serial_in;
            tx.shift_enable = vif.shift_en;
            sr_ap.write(tx);

            // Send actual output to scoreboard
            #1; //delay after posedge
            tx_result = transaction#(NUM_BITS)::type_id::create("tx_result");
            tx_result.parallel_out = vif.parallel_out;
            result_ap.write(tx_result);
        end
    endtask
endclass
```

Figure 4: Shift Register Monitor

pop_front() and compares their parallel_out values. If they match, the pass count is incremented, and the compare-pass message is printed. If values dont match, fail count is incremented, and UVM error is reported.

Finally, during the report_phase, the scoreboard prints the final number of passing and failing comparisons.

```
function void build_phase(uvm_phase phase);
  super.build_phase(phase);
  exp_imp = new("exp_imp", this);
  act_imp = new("act_imp", this);
endfunction
```

Figure 5: Shift Register Scoreboard imp

2.2.5 Predictor

Figure 7 Captures the essence of the Predictor. The predictor is responsible for generating the expected output of the shift register.

In this design, the predictor extends uvm_subscriber, which allows it to receive transactions thorough the write() function. The main variable used is model_q, which represents the Predictor's golden reference model of the Shift Register. On every transaction, the Predictor updates that variable based on reset, as well as shift enable signals.

If nRST is low, the Predictor accurately models the DUT's reset behaviour by reseting the output to all ones. IF reset is not active and shift_enable is high, the predictor shifts in serial_in. The direction of shift depends on the SHIFT_MSB parameter, and the size of parallel_out depends on the NUM_BITS parameter.

After updating its internal golden reference model, the predictor assigns model_q to parallel_out and sends this expected transaction to the scoreboard through predictor_ap

```
function void write_act(transaction#(4) t);
  transaction#(4) temp;
  transaction#(4) exp_tx;
  transaction#(4) act_tx;

  temp = transaction#(4)::type_id::create("temp_act");
  temp.parallel_out = t.parallel_out;

  act_q.push_back(temp);

  `uvm_info("SCB", $sformatf("Received ACTUAL: %s", temp.convert2str()), UVM_HIGH)

  if ((exp_q.size() > 0) && (act_q.size() > 0)) begin
    exp_tx = exp_q.pop_front();
    act_tx = act_q.pop_front();

    if (exp_tx.parallel_out !== act_tx.parallel_out) begin
      fail_count++;
      `uvm_error("SCB",
        $sformatf("COMPARE FAIL | expected parallel_out=%0h actual parallel_out=%0h",
                  exp_tx.parallel_out, act_tx.parallel_out))
    end
    else begin
      pass_count++;
      `uvm_info("SCB",
        $sformatf("COMPARE PASS | parallel_out=%0h", act_tx.parallel_out),
        UVM_LOW)
    end
  end
endfunction

function void report_phase(uvm_phase phase);
  super.report_phase(phase);
  `uvm_info("SCB",
    $sformatf("FINAL RESULTS: PASS=%0d FAIL=%0d", pass_count, fail_count),
    UVM_LOW)
endfunction
```

Figure 6: Shift Register Scoreboard

2.2.6 Sequence

Figure 8 captures a snippet of the Sequence. The Sequence is responsible for generating randomized transaction items that will be sent to the Driver through the Sequencer. In the body() task, a transaction object named req_item is created repeatedly based on the value of trans_c. Each transaction is started using start_item(req_item), randomized using req_item.randomize(), and then completed using finish_item(req_item).

The randomization step generates different values for the transaction fields, such as nRST, shift_enable, and serial_in. If randomization fails, the sequence reports a fatal UVM error. This design allows the testbench to automatically create many randomized shift register scenarios, which improves coverage compared to manually writing each directed test.

3 Evaluation

In this project, the metric used to evaluate the completeness of the testbench is functional coverage. Functional coverage was collected through the Monitor using covergroups, coverpoints, bins, and cross coverage. This metric was chosen because it shows how well the randomized test sequence exercised the important behaviors of the Shift Register, rather than only showing whether the simulation passed or failed.

The coverage model measured toggle coverage as well as functional coverage of control signals. This includes reset behavior, shift enable activity, serial input values, and shift direction cases. Cross coverage was used to check that combinations of these signals were tested together. This is important

```
function void write_act(transaction#(4) t);
  transaction#(4) temp;
  transaction#(4) exp_tx;
  transaction#(4) act_tx;

  temp = transaction#(4)::type_id::create("temp_act");
  temp.parallel_out = t.parallel_out;

  act_q.push_back(temp);

  `uvm_info("SCB", $sformatf("Received ACTUAL: %s", temp.convert2str()), UVM_HIGH)

  if ((exp_q.size() > 0) && (act_q.size() > 0)) begin
    exp_tx = exp_q.pop_front();
    act_tx = act_q.pop_front();

    if (exp_tx.parallel_out !== act_tx.parallel_out) begin
      fail_count++;
      `uvm_error("SCB",
        $sformatf("COMPARE FAIL | expected parallel_out=%0h actual parallel_out=%0h",
                  exp_tx.parallel_out, act_tx.parallel_out))
    end
    else begin
      pass_count++;
      `uvm_info("SCB",
        $sformatf("COMPARE PASS | parallel_out=%0h", act_tx.parallel_out),
        UVM_LOW)
    end
  end
endfunction

function void report_phase(uvm_phase phase);
  super.report_phase(phase);
  `uvm_info("SCB",
    $sformatf("FINAL RESULTS: PASS=%0d FAIL=%0d", pass_count, fail_count),
    UVM_LOW)
endfunction
```

Figure 7: Shift Register Predictor

```
task body();
  transaction#(4) req_item;

  repeat(trans_c) begin
    req_item = transaction#(4)::type_id::create("req_item");
    start_item(req_item);
    if(!req_item.randomize())
      `uvm_fatal("Sequence", "Randomization failed")
    finish_item(req_item);
  end

  `uvm_info("Sequence", $sformatf("Done generation of %0d items", trans_c), UVM_LOW)
endtask: body
```

Figure 8: Shift Register Sequence

because a shift register may work correctly for individual signal values but still fail for certain combinations of reset, shift enable, and serial input, failing due to priority issues.

After running the randomized sequence, the testbench achieved a functional coverage result of **90%** as seen in Figure 9 below. This result indicates that the testbench exercised most of the intended Shift Register scenarios. Any uncovered bins represent cases that were not generated during random testing and could be improved in future work by adding more and more transactions. In this iteration of the testbench, only 20 transactions were added as Questa Sim failed to compile on thinlinc due to cloud issues.

```
uvm_test_top.env.agt.sr_monitor [COV] ================================
uvm_test_top.env.agt.sr_monitor [COV]        Coverage Report
uvm_test_top.env.agt.sr_monitor [COV] ================================
uvm_test_top.env.agt.sr_monitor [COV]   Overall           : 90.00%
uvm_test_top.env.agt.sr_monitor [COV]   cp_nRST           : 100.00%
uvm_test_top.env.agt.sr_monitor [COV]   cp_shift_en       : 100.00%
uvm_test_top.env.agt.sr_monitor [COV]   cp_serial_in      : 100.00%
uvm_test_top.env.agt.sr_monitor [COV]   cx_shift_x_serial: 75.00%
uvm_test_top.env.agt.sr_monitor [COV]   cx_reset_x_shift : 75.00%
uvm_test_top.env.agt.sr_monitor [COV] ================================
0: uvm_test_top.env.scb [SCB] FINAL RESULTS: PASS=19 FAIL=0
```

Figure 9: Functional Coverage Collector

Another way of evaluating the accurateness of the DUT/UVM testbench is through looking at its waveforms. In Figure 10 below, we are able to see that at every single shift high, the correct value of serial in is shifted in. Additionally, when comparing `parallel_out` and `q`, we can see that at every point in time, they are the same.

Another thing to check is negative reset behaviour. When negative reset is asserted, the `parallel_out` and `q` value both reset to all ones.

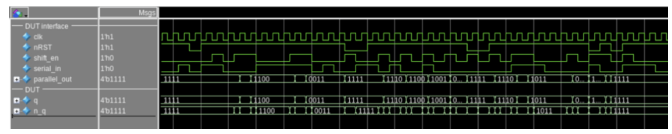

Figure 10: Waveforms

4 Conclusion and Future Work

For future work, this UVM verification environment can be expanded beyond shift registers to support more complex digital designs, such as synchronous and asynchronous FIFOs, counters, FSM-based controllers, and small datapath components. A FIFO would be a natural next step because it introduces important verification challenges, including full and empty conditions, overflow and underflow detection, pointer wraparound, and clock-domain crossing in asynchronous designs.

Additional improvements could include integrating SystemVerilog assertions, directed corner-case tests, and more detailed functional coverage models to strengthen the overall verification quality. These extensions would make the environment more reusable, scalable, and applicable to increasingly complex SoC components in future Purdue SoCET projects.

References

A Teamwork

Everything in this Project was finished by Shawn Chang. Special thanks to Anas and Ayush for helping debug some parts of the waveforms.

Due to the project being an individual project, Ayush has told Digital Library Students to make the report 3-4 pages.